



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 2

по курсу «Защита информации»

на тему: «Разработка электронного аналога Энигмы для шифрования архива»

Студент ИУ7-71Б
(Группа)

(Подпись, дата)

Я. Р. Овчинников
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю. С. Рудинкова
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
РЕАЛИЗАЦИЯ	7
ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ	10

ЗАДАНИЕ

1. Цель работы

Разработка электронного аналога шифровальной машины. Шифрование и расшифровка архивных файлов.

2. Задание

Реализовать в виде программы электронный аналог шифровальной машины «Энигма». Обеспечить шифрование и расшифровку файла архива (tar, zip или др.) с использованием разработанной программы.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1. Основные определения

Информационная сфера — область деятельности, связанная с формированием, хранением, обработкой и использованием информации, а также воздействием информации на общество и человека.

Угроза — потенциальное событие или действие, которое может нанести ущерб информационным активам (кража данных, модификация, уничтожение).

Безопасность — состояние защищённости жизненно важных интересов личности, общества и государства от внутренних и внешних угроз.

Информационная безопасность — состояние защищённости информационной среды, обеспечивающее её формирование, использование и развитие в интересах граждан, организаций и государства.

Шифровальная машина «Энигма» — электромеханическое устройство для шифрования текста, применявшееся Германией во время Второй мировой войны, основанное на многоалфавитной подстановке с использованием роторов.

2. Алгоритм шифрования архивного файла

- 1) Чтение исходного архивного файла (tar, zip или др.) в виде последовательности байтов.
- 2) Инициализация машины «Энигма» с заданными параметрами (начальные позиции роторов, коммутационная панель).
- 3) Последовательная обработка каждого байта файла через механизм «Энигмы»:
 - Прохождение сигнала через коммутационную панель;
 - Прохождение через роторы в прямом направлении;
 - Отражение сигнала рефлектором;
 - Обратное прохождение через роторы;
 - Повторное прохождение через коммутационную панель;

- Поворот роторов после каждого символа.
- 4) Запись зашифрованных байтов в выходной файл.
- 5) Сохранение или передача параметров инициализации для последующей расшифровки.

3. Алгоритм расшифровки архивного файла

- 1) Чтение зашифрованного файла в виде последовательности байтов.
- 2) Инициализация машины «Энигма» с теми же параметрами, что использовались при шифровании (начальные позиции роторов, конфигурация коммутационной панели).
- 3) Последовательная обработка каждого байта через механизм «Энигмы» (процесс идентичен шифрованию, так как «Энигма» является симметричным шифром):
 - Прохождение сигнала через коммутационную панель;
 - Прохождение через роторы в прямом направлении;
 - Отражение сигнала рефлектором;
 - Обратное прохождение через роторы;
 - Повторное прохождение через коммутационную панель;
 - Поворот роторов после каждого символа.
- 4) Запись расшифрованных байтов в выходной архивный файл.
- 5) Проверка целостности полученного архива.

4. Листинг алгоритма работы «Энигмы»

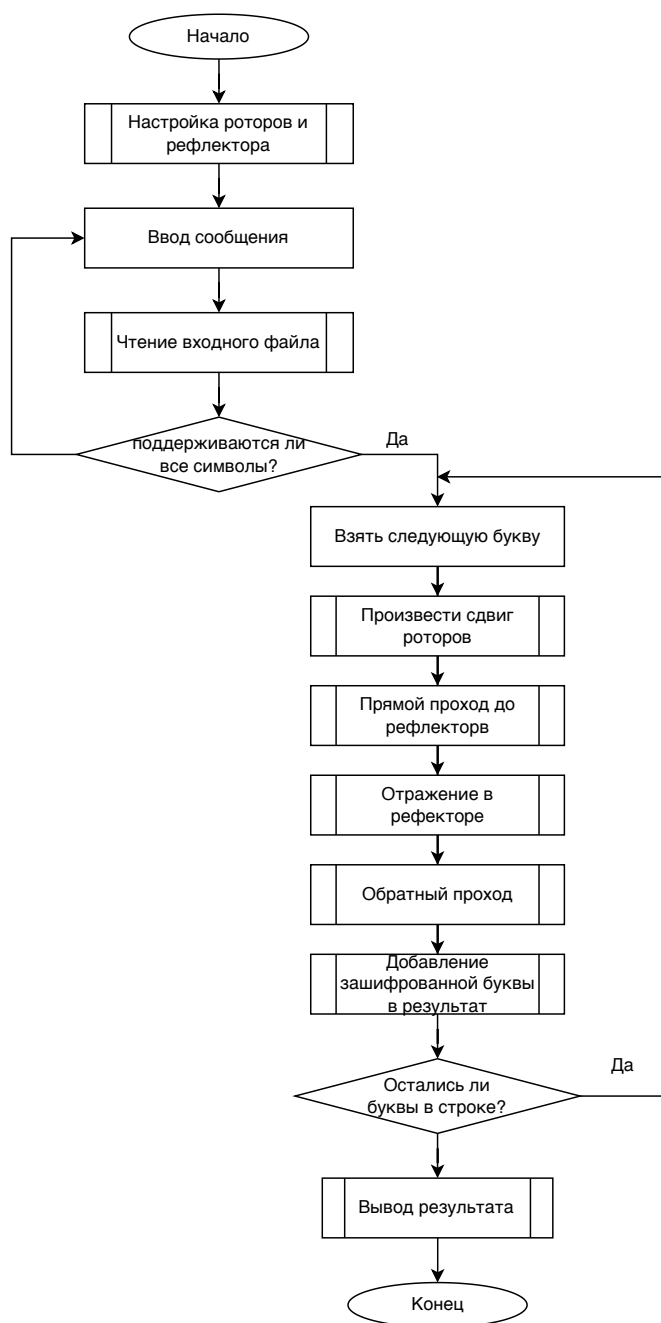


Рисунок 1 – Схема алгоритма работы «Энигмы»

РЕАЛИЗАЦИЯ

На листинге 1 приведена реализация функции для полного цикла шифрования одной буквы.

Листинг 1 – функция полного цикла шифрования одной буквы

```
1 char Enigma::encrypt_letter(char input)
2 {
3     char current_letter = input;
4
5     current_letter = shift_letters(current_letter,
6         rotor_positions[0] - 'A');
7     apply_rotor(active_rotors[0], current_letter);
8
9     for (int i = 1; i < rotor_positions.size(); ++i) {
10         current_letter = shift_letters(current_letter,
11             rotor_positions[i] - rotor_positions[i - 1]);
12         apply_rotor(active_rotors[i], current_letter);
13     }
14
15     current_letter = reflector[current_letter - 'A'];
16     apply_rotor_inverse(active_rotors.back(), current_letter);
17
18     for (int i = active_rotors.size() - 2; i >= 0; --i) {
19         current_letter = shift_letters(current_letter,
20             rotor_positions[i] - rotor_positions[i + 1]);
21         // -(rotor_positions[i + 1] - rotor_positions[i])
22         apply_rotor_inverse(active_rotors[i], current_letter);
23     }
24
25     return shift_letters(current_letter, 'A' -
26         rotor_positions[0]);
27 }
```

На листинге 2 приведена реализация функции для одного шага проворота колец роторов.

Листинг 2 – функция для одного шага проворота колец роторов

```
1 void Enigma::step_rotors()
2 {
3     size_t rotors_number = active_rotors.size();
4     std::vector<int> shifts(rotors_number, 0);
5     shifts[0] = 1;
6
7     for (size_t i = 0; i < rotors_number - 1; ++i) {
8         if (active_rotors[i].second == rotor_positions[i]) {
9             shifts[i + 1] = 1;
10        }
11    }
12
13    for (size_t i = 1; i < rotors_number - 1; ++i) {
14        if (active_rotors[i].second == rotor_positions[i]) {
15            shifts[i] = 1;
16        }
17    }
18
19    for (size_t i = 0; i < rotors_number; ++i) {
20        if (shifts[i] > 0) {
21            rotor_positions[i] =
22                shift_letters(rotor_positions[i], shifts[i]);
23        }
24    }
```

На листинге 3 приведена реализация функции применения проводки при прямом ходе.

Листинг 3 – прямой ход проводки

```
1 void Enigma::apply_rotor(const std::pair<std::string, char>&
2     rotor, char& input)
3 {
4     input = rotor.first[input - 'A'];
5 }
```


На листинге 4 приведена реализация функции применения проводки при обратном ходе.

Листинг 4 – обратный ход проводки

```
1 void Enigma::apply_rotor_inverse(const std::pair<std::string,
  char>& rotor, char& input)
2 {
3     for (int i = 0; i < 26; ++i) {
4         if (rotor.first[i] == input) {
5             input = static_cast<char>('A' + i);
6             return;
7         }
8     }
9 }
```

На листинге 5 приведено программное представление роторов и рефлексоров.

Листинг 5 – хранение роторов и рефлексоров

```
1 const std::unordered_map<uint8_t, std::pair<std::string, char>>
  Enigma::ROTOR_CONFIGS = {{ // <type, <letters_shift, notch>>
2     {1, {"EKMFLGDQVZNTOWYHXUSPAIBRCJ", 'A'}},
3     {2, {"AJDKSIRUXBLHWTMCQGZNPYFVOE", 'B'}},
4     {3, {"BDFHJLCPRTXVZNYEIWGAKMUSQO", 'C'}},
5     {4, {"ESOVZPJAYQUIRHXNLFTGKDCMWB", 'R'}},
6     {5, {"VZBRGITYUPSDNHLXAWMJQOFECK", 'H'}}
7 }};
8
9 const std::unordered_map<char, std::string>
  Enigma::REFLECTOR_CONFIGS = {{
10     {'A', "EJMZALYXVBWFCRQUONTSPIKHGD"},
11     {'B', "YRUHQSLDPXNGOKMIEBFZCWVJAT"},
12     {'C', "FVPJIAOYEDRZXWGCTKUQSBNMHL"}
13 }};
```

ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

На картинках ниже продемонстрированы основные сценарии работы программы

```
lab1 git:(main) x ./enigma
=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 
```

Рисунок 1 – Основное меню программы

```
=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 2

Доступные роторы:
Ротор 1: EKMFLGDQVZNTOWYHXUSPAIBRCJ | Notch: A
Ротор 2: AJDKSIRUXBLHWTMCQGZNPYFVOE | Notch: B
Ротор 3: BDFHJLCPRTXVZNYEIWGAKMUSQO | Notch: C
Ротор 4: ESOVPZJAYQUIRHXLNFTGKDCMWB | Notch: R
Ротор 5: VZBRGITYUPSDNHLXAWMJQOFECK | Notch: H

Введите номера роторов установки в порядке следования к рефлектору (1-5, через пробел): 1 3 2
Введите позиции колец для 3 ротора(-ов) (A-Z, через пробел): A A A
Введите тип рефлектора (A, B, C): A
Машина настроена!
=== Текущая конфигурация ===
Роторы:
Позиция 1: Барабан: EKMFLGDQVZNTOWYHXUSPAIBRCJ | Notch: A | Ring: A
Позиция 2: Барабан: BDFHJLCPRTXVZNYEIWGAKMUSQO | Notch: C | Ring: A
Позиция 3: Барабан: AJDKSIRUXBLHWTMCQGZNPYFVOE | Notch: B | Ring: A
Рефлектор: Барабан: EJMZALYXVBWFCRQUONTSPIKHGD | Тип: A
```

Рисунок 2 – Меню настройки Машины "Энигма"

```

=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 1

Выберите способ ввода данных:
1. Прочитать текстовое сообщение из файла (.txt)
2. Ввести сообщение с клавиатуры
3. Работа с архивом
0. Вернуться в главное меню
Ваш выбор: 3

Выберите операцию с архивом:
1. Зашифровать архив
2. Расшифровать архив
0. Вернуться в предыдущее меню
Ваш выбор: 1

Доступные архивные файлы в директории out/:
1. 123.zip
2. 123_decoded.zip

Выберите архив (введите номер от 1 до 2): 1
Выбран файл: 123.zip
Файл прочитан (1330 байт)
Выполняется шифрование...
Архив успешно зашифрован и сохранен как: out/123.zip_encoded
Размер зашифрованного файла: 1330 байт

```

Рисунок 3 – Результат шифрования архива

```

=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 3

=== Текущая конфигурация ===
Роторы:
Позиция 1: Барабан: EKMFLGDQVZNTOWYHXUSPAIBRCJ | Notch: A | Ring: Z
Позиция 2: Барабан: AJDKSIRUXBLHWTMCQGZNPYFVOE | Notch: B | Ring: F
Позиция 3: Барабан: BDFHJLCPRTXVZNYEIWGAKMUSQO | Notch: C | Ring: A
Рефлектор: Барабан: EJMZALYXVBWFCRQUONTSPIKHGD | Тип: A

```

Рисунок 4 – Состояние машины после выполнения шифрования

```

=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 1

Выберите способ ввода данных:
1. Прочитать текстовое сообщение из файла (.txt)
2. Ввести сообщение с клавиатуры
3. Работа с архивом
0. Вернуться в главное меню
Ваш выбор: 3

Выберите операцию с архивом:
1. Зашифровать архив
2. Расшифровать архив
0. Вернуться в предыдущее меню
Ваш выбор: 2

Доступные зашифрованные файлы в директории out/:
  1. 123.zip_encoded

Выберите зашифрованный файл (введите номер от 1 до 1): 1
Выбран файл: 123.zip_encoded
Зашифрованный файл прочитан (1330 байт)
Выполняется расшифровка...
Архив успешно расшифрован и сохранен как: out/123_decoded.zip
Размер расшифрованного файла: 1330 байт

```

Рисунок 5 – Результат расшифровки архива

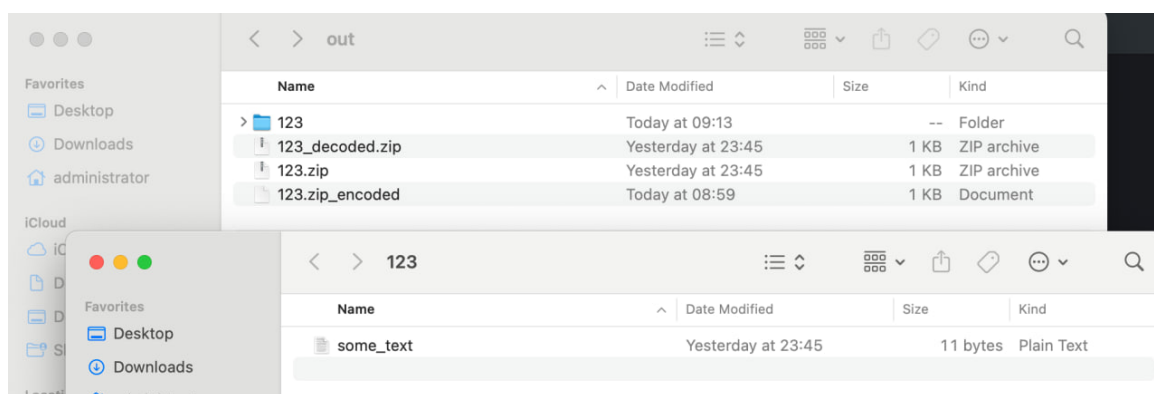


Рисунок 6 – Открытие расшифрованного архива

Также была предусмотрена обработка большинства ошибочных ситуаций без экстренного завершения программы:

```
lab1 git:(main) x ./enigma
=== Машина Энигма ===
1. Ввести сообщение
2. Изменить конфигурацию
3. Посмотреть конфигурацию
4. Сбросить машину к начальным позициям
0. Выход
> 1
Машина ещё не настроена. Сначала настройте конфигурацию
```

Рисунок 7 – Обработка ошибочных ситуаций